MIVA OPEN UNIVERSITY · SCHOOL OF POSTGRADUATE STUDIES · MASTER OF INFORMATION TECHNOLOGY

MIT 8103 Advanced Database Systems: Portfolio Documentation

Student: **Amazobi Amarachi** Student ID: **301919110** Assessment: **First Semester CA, September Cohort 2026**

GitHub: **https://github.com/AmazobiAmarachi/301919110-advanced-database-portfolio**

1. Selected case study: PayNaija Wallet

PayNaija Wallet is a simulated Nigerian digital wallet and payments company (fintech) operating in eight states and subject to the Nigeria Data Protection Act (NDPA) 2023. **Nature of business:** retail digital payments covering wallets, savings, person-to-person transfers, POS merchant payments, airtime and bills, through app, USSD and web. **Database users:** customers (via the app), support staff, KYC/compliance officers, fraud analysts, administrators and management. **Major data:** customers, KYC documents, accounts and balances, cards, merchants, transfers, a ledger of transactions, regions, staff and an audit log. **Key operations:** register and verify customers, open accounts, deposit/withdraw, **transfer funds**, **reverse transfers**, generate statements, **audit account balances**, detect fraud and produce regional reports. All work was done on my Windows 11 laptop (PostgreSQL 16, pgAdmin 4, Docker Desktop) using synthetic data.

2. Portfolio activities

Portfolio 1: Database Design and Modelling

I followed the four design stages (requirements → ER model → logical schema → physical design). The ER model has 10 entities; the recursive many-to-many relationship "account sends money to account" is resolved by the `TRANSFERS` associative entity. A flat customer-statement spreadsheet was normalised step by step through 1NF, 2NF (removing the partial dependency on `account_number`) and 3NF/BCNF (removing customer → region → zone and merchant → category). Integrity is enforced inside the database: PK/FK, UNIQUE, `CHECK (balance >= 0)`, a Nigerian phone-format check, an 18+ rule, `ON DELETE RESTRICT`, an audit trigger and least-privilege roles. I loaded 2,000 customers, 2,987 accounts and 200,000 ledger rows; **all 7 invalid operations in my constraint tests were rejected**.

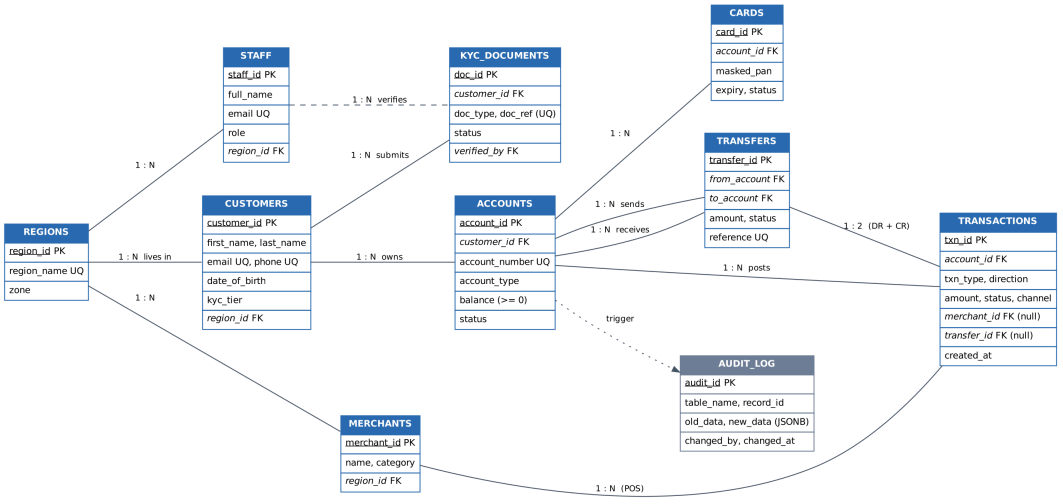


Figure 1. PayNaija ER diagram (PK = primary key, FK = foreign key, UQ = unique)

Portfolio 2: Query Processing and Optimisation

Seven business queries were timed with `EXPLAIN (ANALYZE, BUFFERS)` (median of 5 runs) before and after a composite B+ tree index, a partial covering index, a foreign-key index, a sargable rewrite and a materialised view.

Query	Before	After	Plan change	Gain
Q1 account statement	45.05 ms	0.12 ms	Parallel Seq Scan → Bitmap Index Scan (cost 5,617 → 65)	**≈366×**
Q2 customer statement (3-table join)	46.86 ms	0.47 ms	Full scan → Nested Loop of three Index Scans	**≈99×**
Q3 failed txns: function → range rewrite	48.32 ms	0.16 ms	Partial index used for both conditions	**≈312×**
Q6 fraud velocity	45.72 ms	0.37 ms	Seq Scan → partial index	**≈123×**
Q4 regional report (OLAP)	119.17 ms	119.97 → 0.13 ms (MV)	Index correctly ignored; materialised view used	**≈931×**

Analysis: the cost-based optimiser used the indexes only for highly selective OLTP queries and correctly ignored them for reports that read most of the table, which needed pre-aggregation instead. The plans also showed that about 38 ms of my 45 ms baseline was the cost of starting a parallel worker on Windows. Indexes made inserts about 29% slower (471.7 ms vs 364.9 ms per 50,000 rows), an acceptable trade-off for a read-heavy wallet.

Portfolio 3: Transactions and Concurrency

I demonstrated COMMIT (₦10,000 transfer with audit rows), automatic ROLLBACK when a debit violated `balance >= 0` after the credit had already run (atomicity), SAVEPOINT, and a `transfer_funds()` procedure. Two simultaneous sessions reproduced a **lost update** (₦40,000 left instead of ₦35,000) with no error raised. It was fixed with `SELECT … FOR UPDATE` and, separately, with REPEATABLE READ (*"could not serialize access"*, then a retry). A non-repeatable read (50,000 → 49,000) appeared under READ COMMITTED but not REPEATABLE READ, and a **deadlock** was detected and resolved. For the brief's fintech operations, I added `reverse_transfer()` (a compensating transaction: transfer 325 reversed, balances restored, double reversal blocked) and five ledger audit checks, all passing (credits = debits = ₦62,000; difference ₦0.00).

pgbench, 8 clients × 40 transfers	Deadlock failures	Throughput	Money conserved
Naive (locks in transfer direction)	254 / 320 (79.4%)	0.38 tps	Yes
`transfer_funds()` (locks lowest account id first)	**0 / 320 (0%)**	**833.3 tps**	Yes

Portfolio 4: NoSQL and Advanced Data Models

Using polyglot persistence, money stays in ACID PostgreSQL, and three workloads move to NoSQL models running in Docker, all built from data exported from my own PostgreSQL. **Key-value (Redis):** sessions with a 900 s TTL, a PIN brute-force limiter (4th attempt blocked), cache-aside balances and a sorted-set leaderboard. **Document (MongoDB):** 2,000 customer documents with KYC, devices and preferences *embedded*, and 20,000 transactions *referenced*; a `$jsonSchema` validator; an aggregation pipeline per region; and `explain` showing **COLLSCAN of 20,000 documents → IXSCAN of 7**. Summing balances stored as JSON doubles produced ₦22,755,982.259999998, which is real evidence for my NUMERIC design choice. **Graph (Neo4j):** `(a)-[:SENT*3..5]->(a)` found the ring 500→600→700→800→500 with amounts shrinking by ₦5,000 per hop (layering), and all four owners shared one device.

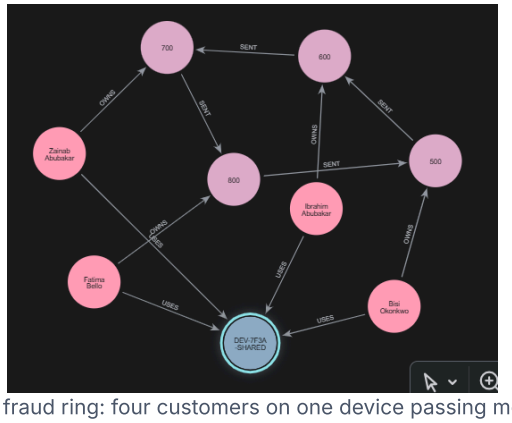


Figure 2. Neo4j fraud ring: four customers on one device passing money in a circle

Portfolio 5: Distributed and Cloud Database Exercise

I built PostgreSQL 16 **streaming replication** between a "Lagos" primary and an "Abuja" hot standby as two Docker containers (`pg_basebackup` with a replication slot), then loaded my PayNaija database, which replicated automatically.

Evidence: `streaming` with a 2.4 ms replay lag, a new customer readable on the standby within a second, writes rejected on the standby, and identical counts (2,001 customers, 201,438 transactions). **CAP in practice:** with Abuja stopped (a partition), synchronous replication blocked a deposit on `SyncRep` for 5+ seconds (consistency over availability, CP), while asynchronous replication accepted it in 5.5 ms with a risk of loss (availability, eventual consistency). **Failover:** `docker kill` crashed Lagos; Abuja was promoted and processed transfer 721 in 7.6 ms with all 720 earlier transfers intact. Horizontal fragmentation by region showed partition pruning, plus a hash-sharding plan and a Multi-AZ cloud design with NDPA data residency.

3. Individual technical reflection

Design choices. My most important decision was storing money as `NUMERIC(15,2)` rather than FLOAT, with `CHECK (balance >= 0)` enforced by the database itself. I first chose this from theory, but Portfolio 4 proved it: when MongoDB summed the same balances stored as floating-point numbers, Anambra's total came back as ₦22,755,982.259999998. In MongoDB I embedded KYC and devices (always read with the customer) but referenced transactions, because they grow without limit.

Challenges. Several problems came from my environment rather than the databases. My screenshots kept disappearing because Windows Explorer and Paint saved into my work OneDrive while my repository lives in a different Documents folder; I solved it by opening folders from the terminal and copying captures straight into the repo with a small shell function. A push was rejected after I edited a README on the GitHub website, which taught me to fetch and rebase, and to keep one source of truth. In a small way it was the same consistency problem as two diverged replicas. In Portfolio 2 my baseline queries took about 45 ms; reading the plan showed the scan itself took only 8 ms, and the rest was the cost of starting a parallel worker process on Windows. The naive deadlock load test took minutes because each deadlock waits one second before detection, and seeing 79% of transfers fail made the value of a fixed lock order very clear. The Linux replication scripts could not run on Windows, so I rebuilt Portfolio 5 with Docker containers.

Lessons learned. The most surprising lesson was that CAP is a configuration decision: one setting, `synchronous_standby_names`, moved the same PostgreSQL system between CP and AP behaviour. I also learned that indexes are not a universal fix (my regional report did not change until I used a materialised view), and that the most dangerous concurrency bug, the lost update, raises no error at all. For a fintech, money must be ACID and strongly consistent, while sessions, profiles and fraud analysis benefit from key-value, document and graph models.

4. References

MIVA Open University (2025) *MIT 8103 Advanced Database Systems*, Study Sessions 1–6. · PostgreSQL Global Development Group (2025) *PostgreSQL 16 Documentation*: Ch. 13 Concurrency Control, Ch. 14 Performance Tips, Ch. 27 High Availability, Load Balancing and Replication. · MongoDB Inc. (2025) *MongoDB Manual: Data Modeling; Explain Results*. · Neo4j Inc. (2025) *Cypher Manual*. · Redis Ltd. (2025) *Redis Documentation*. · Docker Inc. (2025) *Docker Desktop for Windows; Docker Hub official images postgres, mongo, redis, neo4j*. · Elmasri, R. & Navathe, S. (2016) *Fundamentals of Database Systems*, 7th edn, Pearson. · Federal Republic of Nigeria (2023) *Nigeria Data Protection Act 2023*. · All data is synthetic, generated by `02_seed.sql`; no real personal data was used.

5. AI use declaration

Tool used	Claude (Anthropic)
Used for	Course summary for revision; planning the portfolio; drafting SQL, shell, MongoDB and Cypher scripts and adapting them to Windows/Docker; explaining execution plans and errors while I worked; drafting README text, this documentation and the reflection from my own results and experiences.
How I verified it	I installed every tool on my own laptop and ran every script myself; all timings, plans, outputs and screenshots come from my machine and were committed progressively to GitHub. I checked each result against the theory, corrected the READMEs so they report my own numbers, and read and edited the drafted text, including the reflection, so it describes what actually happened. I can explain every script, design decision and result.